**SERVICIO NACIONAL DE APRENDIZAJE – SENA**

Centro de la Industria, la Empresa y los Servicios

Tecnólogo en Análisis y Desarrollo de Software · Ficha 3145555

## PROYECTO DRIVIQUE

# Documentación de Propuesta Técnica del Software

Sistema integral de gestión y alquiler de vehículos

**Integrantes**

Laura Vanessa Pérez Perdomo

Danna Valentina Barrios Penagos

Emily Sharith Amézquita Saavedra

**2026**

# Contenido

1. Introducción
2. Resumen ejecutivo y alcance del producto
3. Objetivos técnicos
4. Pila tecnológica
5. Arquitectura del sistema
6. Patrones de diseño implementados
7. Modelo de datos
8. Seguridad y autorización
9. Integraciones externas
10. Calidad, pruebas e integración continua
11. Experiencia de usuario, idiomas y accesibilidad
12. Requisitos no funcionales de referencia
13. Riesgos técnicos y mitigaciones
14. Hoja de ruta de evolución
15. Conclusiones
Referencias

# 1. Introducción

Este documento presenta la propuesta técnica de **Drivique**, un sistema integral de gestión y alquiler de vehículos. Describe los componentes del producto, las tecnologías elegidas y su justificación, la arquitectura, los patrones de diseño, el modelo de datos y el modelo de seguridad. Complementa la Especificación de Requisitos de Software (SRS) del proyecto, que define el qué del sistema, y se articula con el documento de acciones correctivas, preventivas y de mejoramiento (CAPA), que registra cómo se ha ido corrigiendo y fortaleciendo la solución.

El público objetivo es el equipo de desarrollo, el instructor evaluador y cualquier persona que deba mantener, auditar o extender el software.

### Control del documento

| | |
|---|---|
| **Proyecto** | Drivique |
| **Documento** | Propuesta técnica del software |
| **Versión** | 1.0 |
| **Estado** | Vigente |
| **Documentos relacionados** | SRS Drivique Consolidado 2026 · Documento de acciones correctivas, preventivas y de mejoramiento (CAPA) |

# 2. Resumen ejecutivo y alcance del producto

Drivique está estructurado como un sistema **multicanal y multirol**. El cliente final usa una aplicación móvil para consultar la flota, cotizar en tiempo real, reservar, pagar y firmar contratos, mientras que los empleados y administradores operan la empresa desde un panel web. Ambos canales consumen una única API que concentra las reglas de negocio, los precios, la autenticación y la persistencia.

### Tabla 1. Componentes del sistema

| Componente | Repositorio | Usuarios | Responsabilidad |
|---|---|---|---|
| App móvil | `app-drivique` | Cliente final | Autoservicio: consulta de flota, cotización en tiempo real, reserva guiada, pagos digitales y presenciales, firma de contratos y gestión del perfil KYC. |
| Panel web | `web-drivique` | Empleados, `BRANCH_ADMIN`, `SUPER_ADMIN` | Gestión operativa, aislamiento estricto de datos por sede, auditoría y exportación de reportes. |
| Backend API | `drivique-api` | Apps móvil y web | Núcleo empresarial en Java y Spring Boot; fuente de verdad de reglas de negocio, precios, autenticación y persistencia. |
| Base de datos | PostgreSQL 17 | API | Modelo relacional multiesquema para segregar dominios de negocio. |

**Fuera de alcance:** facturación electrónica con validez fiscal, contabilidad avanzada y operación sin conexión, conforme a las limitaciones definidas en el SRS.

## 3. Objetivos técnicos

- Garantizar que la API sea la **única fuente de verdad** para reglas de negocio, precios y permisos.
- Mantener una arquitectura **modular y por capas** que facilite el mantenimiento y la incorporación de nuevas funciones.
- Aislar los datos por sucursal para que cada encargado vea y opere solo lo de su sede.
- Soportar pagos digitales y presenciales mediante un diseño extensible a nuevos medios de pago.
- Proteger los datos personales y documentales cumpliendo los principios de Habeas Data.
- Validar el software con pruebas de integración sobre un entorno idéntico al de producción.

## 4. Pila tecnológica

La Tabla 2 resume las tecnologías seleccionadas, su versión y la razón técnica de cada elección.

**Tabla 2. Matriz de tecnologías**

| Dominio | Tecnología | Versión | Justificación técnica |
|---|---|---|---|
| Backend | Java / Spring Boot | Java 21 / Spring Boot 3.3 | Rendimiento empresarial, tipado estricto, gestión de transacciones y seguridad robusta. |
| Persistencia y ORM | Spring Data JPA / Hibernate | 6.x | Abstracción del acceso a datos con mapeo relacional de entidades. |
| Base de datos | PostgreSQL | 17.x | Motor relacional con soporte nativo de esquemas múltiples, restricciones y JSONB. |
| Panel web | React / Vite | React 18 / Vite 5 | SPA de alta velocidad, recarga instantánea en desarrollo, arquitectura por hooks y CSS propio con colores HSL. |
| App móvil | React Native / Expo | Expo SDK 51 / Expo Router 3 | Aplicación nativa multiplataforma con enrutamiento basado en archivos. |
| Lenguaje móvil | TypeScript | 5.3 (modo estricto) | Detecta errores de tipo en compilación, útil para contratos e interfaces. |
| Seguridad | Spring Security + JWT | OAuth2 / PKCE | Tokens firmados, Refresh Tokens y autenticación social (Google y Facebook). |

| Dominio | Tecnología | Versión | Justificación técnica |
|---|---|---|---|
| Pruebas y CI | Testcontainers + JUnit 5 | PostgreSQL 17 Alpine | Pruebas de integración reales sobre contenedores idénticos a producción. |

# 5. Arquitectura del sistema

## 5.1 Vista global

La Figura 1 muestra la arquitectura. Los clientes web y móvil se comunican con `drivique-api` por HTTPS, en formato JSON y con el encabezado `Authorization: Bearer <JWT>`. La API aplica seguridad y reglas de negocio, accede a PostgreSQL mediante JPA e invoca servicios externos como Wompi y los proveedores de identidad social.

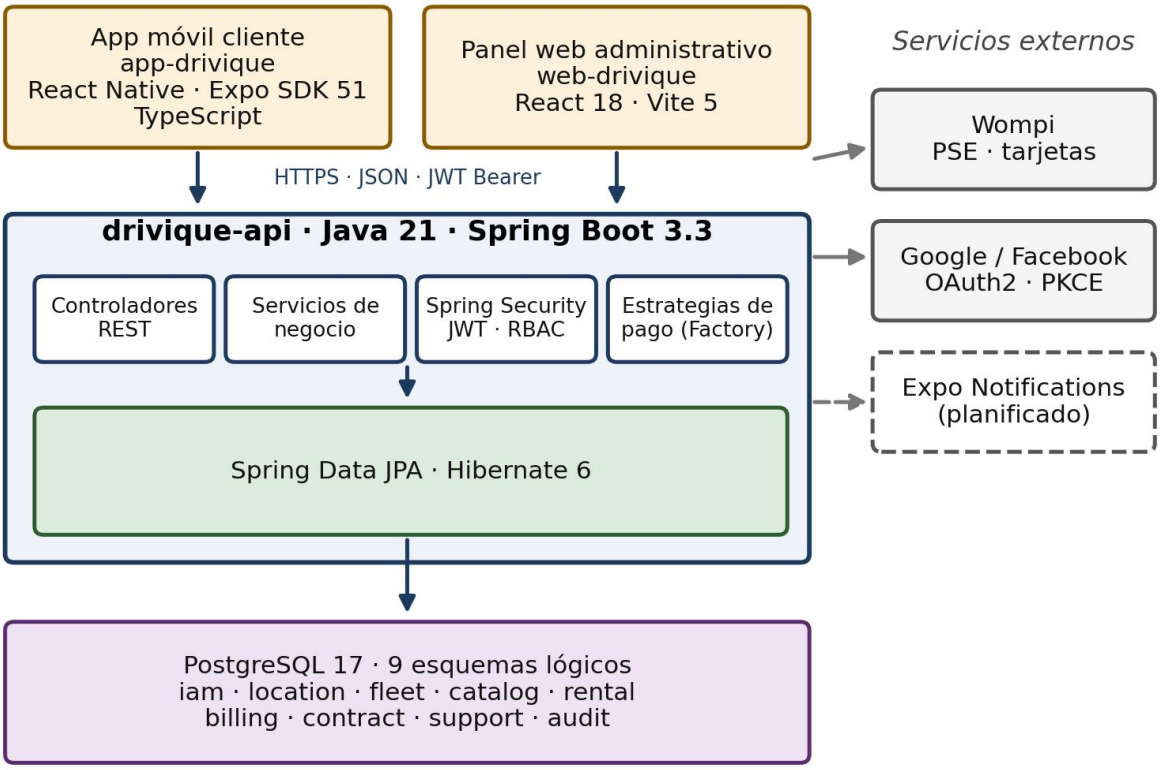



*Figura 1. Arquitectura global de Drivique.*

## 5.2 Organización interna

Tanto el código web como el móvil siguen una **arquitectura modular por funcionalidad (feature-first)**: cada módulo es autocontenido y agrupa sus pantallas, hooks y servicios. Los módulos principales son `auth`, `catalog`, `reservation`, `payments`, `profile` y `admin`.

Dentro de cada módulo se aplica una **arquitectura por capas** con responsabilidades separadas:

1. **UI (vistas):** componentes y pantallas, sin lógica de negocio.

2. **Lógica de estado (custom hooks):** coordina el estado y las reglas de presentación.
3. **Capa de servicios:** encapsula las llamadas a la API y las transformaciones de datos.
4. **Infraestructura / API:** cliente HTTP, almacenamiento seguro de tokens y configuración.

## 6. Patrones de diseño implementados

**Tabla 3. Patrones y su aplicación**

| Patrón | Aplicación en Drivique | Beneficio |
|---|---|---|
| Custom Hooks | Web: `useBranchScope` (restricción por sede), `useManagementTable` (búsqueda y paginación), `useManagementExport` (Excel y PDF).<br>Móvil: `useAuth`, `useCatalog`, `useProfile`, `useIdioma` (5 idiomas), `useCurrency` (COP, USD, EUR). | Desacopla la lógica de la presentación y permite reutilizarla. |
| Strategy y Factory Method | La interfaz `PaymentStrategy` se implementa en `WompiPaymentStrategy` (PSE y tarjetas) y `CashPaymentStrategy` (cobro en caja de sede). `PaymentProcessorFactory` instancia la estrategia según el método elegido. | Agregar un medio de pago no obliga a modificar el flujo existente. |
| Wizard / Step | La reserva móvil es un flujo de tres pasos: 1) fechas y sede, 2) seguros y extras, 3) conductor y pago. | Validación progresiva y menor carga cognitiva para el usuario. |
| Adapter | La generación de contratos y comprobantes usa `pdfService.ts` en móvil y `jsPDF` en web. | El mismo documento se adapta a cada motor de impresión. |
| Observer / Pub-Sub | `AuthContext`, `ThemeContext` y `LanguageContext` (React Context API) notifican cambios de sesión, tema e idioma. | Estado global reactivo sin acoplamiento entre pantallas. |

## 7. Modelo de datos

La base de datos PostgreSQL 17 se organiza en **nueve esquemas lógicos** que separan los dominios de negocio (Figura 2 y Tabla 4). Esta segregación mejora el orden, facilita los permisos por dominio y reduce el riesgo de acoplamiento entre módulos.

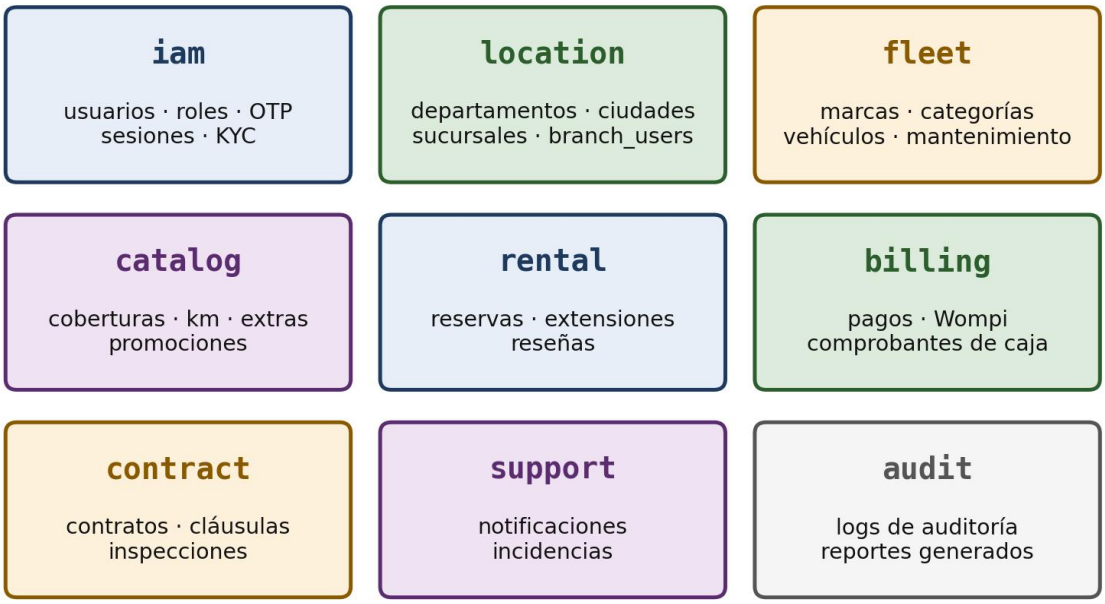



*Figura 2. Esquemas lógicos de la base de datos.*

**Tabla 4. Contenido de cada esquema**

| Esquema | Contenido |
|---|---|
| `iam` | Usuarios, roles, permisos, sesiones, códigos OTP, cuentas sociales y datos KYC. |
| `location` | Departamentos, ciudades, sucursales y personal asignado (`branch_users`). |
| `fleet` | Marcas, categorías, vehículos, transmisiones, combustibles, mantenimiento y favoritos. |
| `catalog` | Coberturas de seguro, planes de kilometraje, servicios adicionales y promociones. |
| `rental` | Reservas, puntos de entrega, extensiones y reseñas. |
| `billing` | Pagos, transacciones Wompi, comprobantes de caja y métodos de pago guardados. |
| `contract` | Contratos de alquiler, cláusulas e inspecciones de entrega y devolución. |
| `support` | Notificaciones y reportes de incidencias mecánicas. |
| `audit` | Registros de auditoría administrativa y reportes generados. |

# 8. Seguridad y autorización

## 8.1 Control de acceso basado en roles (RBAC)

**Tabla 5. Roles del sistema**

| Rol | Canal | Alcance |
|---|---|---|
| `SUPER_ADMIN` | Panel web | Acceso global a todas las sedes, configuración, usuarios y auditoría. |
| `BRANCH_ADMIN` | Panel web | Administrador de sede: opera solo los registros de su `branchId`. |

| Rol | Canal | Alcance |
|---|---|---|
| `EMPLOYEE` | Panel web | Operario de sede: tareas de entrega, recepción y caja dentro de su sede. |
| `CUSTOMER` | App móvil | Cliente final: reservas, pagos, contratos y perfil propios. |

### 8.2 Aislamiento por sucursal

Los usuarios con rol `BRANCH_ADMIN` o `EMPLOYEE` solo pueden ver y operar registros de la sucursal que tienen asignada. En el panel web esta regla se aplica con el hook `useBranchScope`; en el servidor se valida de nuevo, porque la restricción del cliente nunca se considera suficiente.

### 8.3 Autenticación y protección de endpoints

- Autenticación mediante **JWT firmados** y **Refresh Tokens** para renovar la sesión sin pedir credenciales de nuevo.
- Inicio de sesión social con **Google y Facebook** (OAuth2 con PKCE) y verificación por código OTP.
- Cada solicitud HTTP exige el encabezado `Authorization: Bearer <JWT>`; el servidor valida rol y permisos en cada petición.
- Cifrado del canal con HTTPS/TLS y almacenamiento de contraseñas con hash.

### 8.4 Protección de datos personales

Al eliminar una cuenta, el sistema purga los métodos de pago guardados y los documentos KYC, y conserva únicamente registros transaccionales anonimizados cuando existe historial operativo. El detalle de esta decisión se documenta en el documento CAPA (incidencia 2 y acción preventiva B).

## 9. Integraciones externas

**Tabla 6. Servicios integrados**

| Servicio | Uso | Estado |
|---|---|---|
| Wompi | Pagos digitales con PSE y tarjetas; confirmación de transacciones. | Integrado. El listener de webhooks en vivo está planificado a corto plazo. |
| Google y Facebook | Autenticación social (OAuth2 con PKCE). | Integrado. |
| Correo electrónico | Envío de códigos OTP y notificaciones transaccionales. | Según SRS. |
| Expo Notifications | Avisos móviles de expiración de reserva y entrega del vehículo. | Planificado a corto plazo. |

## 10. Calidad, pruebas e integración continua

- **Pruebas de integración con Testcontainers:** se usa la imagen oficial `postgres:17-alpine`, de modo que ninguna prueba se considera válida si no corre sobre PostgreSQL 17 real. Así se evitan las diferencias de dialecto de bases en memoria como H2.
- **JUnit 5** como marco de pruebas, con limpieza explícita de tablas por esquema antes de cada prueba y datos con nombres únicos para garantizar aislamiento.
- **GitHub Actions** como integración continua. El plugin `maven-surefire-plugin` se configura con `-DtrimStackTrace=false` y el flujo `.github/workflows/ci.yml` vuelca la cadena *Caused by* en las anotaciones del job para acelerar el diagnóstico.
- **TypeScript en modo estricto** en la app móvil para detectar errores de tipo antes de ejecutar.

## 11. Experiencia de usuario, idiomas y accesibilidad

- **Cinco idiomas** en la app móvil mediante `useIdioma`, y **tres monedas** (COP, USD y EUR) mediante `useCurrency`.
- **Tema claro y oscuro** gestionado por `ThemeContext`.
- Generación de **contratos y comprobantes en PDF** en ambos canales.
- Cumplimiento de los lineamientos **WCAG 2.1 nivel AA** definidos en el SRS, con interfaces simples, consistentes y adaptables.

## 12. Requisitos no funcionales de referencia

Las metas siguientes provienen del SRS y orientan las pruebas de aceptación del producto. Se miden en la fase de pruebas de carga y de seguridad.

**Tabla 7. Metas no funcionales**

| Atributo | Meta |
|---|---|
| Rendimiento | Respuesta menor de 2 s en peticiones comunes; hasta 1.000 solicitudes por segundo; confirmación de reserva en menos de 3 s. |
| Seguridad | HTTPS/TLS 1.3, JWT, contraseñas con BCrypt, lineamientos OWASP Top 10 y Habeas Data. |
| Disponibilidad | 99,9 % anual con redundancia y balanceo de carga. |
| Mantenibilidad | Capas desacopladas, principios SOLID y documentación de endpoints con OpenAPI. |
| Resiliencia | Reintentos con retroceso exponencial y recuperación de transacciones fallidas. |

## 13. Riesgos técnicos y mitigaciones

**Tabla 8. Matriz de riesgos**

| Riesgo | Impacto | Mitigación |
|---|---|---|
| Indisponibilidad de la pasarela de pagos | El cliente no puede pagar en línea. | Estrategia de pago en efectivo en caja de sede como alternativa. |
| Diferencias entre entorno de pruebas y producción | Fallos que solo aparecen en producción. | Testcontainers con PostgreSQL 17 real en CI. |
| Conservación indebida de datos sensibles | Incumplimiento de Habeas Data. | Purga y anonimización al eliminar la cuenta. |
| Carga de lectura sobre PostgreSQL | Lentitud en catálogo y consultas frecuentes. | Caché distribuido con Redis (mediano plazo). |
| Archivos KYC en almacenamiento local | Escalabilidad y seguridad limitadas. | Migración a S3 o MinIO con URLs firmadas. |
| Diagnóstico lento de fallos en CI | Mayor tiempo de corrección. | Trazas completas y anotaciones automáticas en GitHub Actions. |

## 14. Hoja de ruta de evolución

El plan de mejoramiento se detalla en el documento CAPA. En resumen:

- **Corto plazo:** webhooks de confirmación en vivo de Wompi y notificaciones push móviles con Expo.
- **Mediano plazo:** caché con Redis y almacenamiento S3 o MinIO para documentos KYC.
- **Largo plazo:** observabilidad con OpenTelemetry, Prometheus y Grafana, y políticas automáticas de retención legal de documentos.

## 15. Conclusiones

La propuesta técnica de Drivique combina una pila moderna y madura (Java 21, Spring Boot 3.3, PostgreSQL 17, React y Expo) con decisiones de diseño que priorizan la mantenibilidad: arquitectura modular por capas, patrones de diseño claros, esquemas de datos segregados y un modelo de seguridad basado en roles con aislamiento por sede. Las pruebas sobre PostgreSQL real y la integración continua reducen el riesgo de regresiones, y la hoja de ruta identifica las mejoras de rendimiento, observabilidad y cumplimiento legal que consolidarán el producto.

## Referencias

[1] Equipo Drivique, “Especificación de Requisitos de Software – Drivique Consolidado,” SENA, 2026.

[2] IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[3] OWASP Foundation, “OWASP Top 10:2021.” https://owasp.org/Top10

[4] W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” 2018. https://www.w3.org/TR/WCAG21
[5] M. Jones, J. Bradley y N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
[6] Wompi, “Documentación oficial de la API.” https://docs.wompi.co